

Framing Sections	Description and commitments to be filled by the student
1. Project Identity & Repository	Project: DevSecOps CI/CD Platform & AI Threat Detection (SecOps-Pipeline) Student: AMHIDI Hamza Repository: https://github.com/echoenvoy/SecOps-Pipeline
2. Existing Baseline Summary	A database-backed web application featuring user authentication, protected REST API endpoints, and a web dashboard, containerized with Docker. Currently functional and deployable in a local Kubernetes cluster (kind-devsecops-lab) connected to an isolated bridge network (devsecops-net).
3. Target Audience & Potential Client	DevSecOps engineers, Blue Team SOC analysts, and security leads in cloud-native organizations seeking to eliminate code vulnerabilities prior to release while continuously detecting and triaging runtime attacks in real-time.
4. Problem Statement & Client Need	Application security is frequently decoupled from runtime operations: vulnerable code slips past manual reviews, while production environments generate overwhelming volumes of un-correlated logs. This leads to SOC alert fatigue, missed behavioral attacks (brute force, IDOR, privilege escalation), and slow incident response
5. Innovation Vision (S5)	Connects shift-left pre-deployment security gates (Semgrep SAST, SonarQube quality gates) with continuous shift-right runtime surveillance (Wazuh HIDS, ELK Stack, TheHive) and local AI-driven log anomaly detection. This ensures known vulnerabilities are blocked in CI/CD, while runtime threats are automatically scored and escalated into actionable incident cases.
6. Artificial Intelligence	Behavioral Log Anomaly Detection. An unsupervised AI pipeline ingests authentication and HTTP access logs from ELK/Wazuh, extracts temporal and request

Component	features (request frequency, IP reputation, parameter entropy), flags abnormal behavior (credential stuffing, IDOR probing, enumeration), and outputs anomaly confidence scor
7. Local AI Stack (Excluding APIs)	Python, Scikit-learn (Isolation Forest & One-Class SVM for tabular log anomaly detection), PyTorch (LSTM Autoencoder for sequential log anomaly detection), and Pandas/NumPy for feature extraction. All training and inference run 100% locally on-premise without external cloud APIs.
8. Second Engineering Domain	Cybersecurity & Cloud/DevOps Technologies: Docker, Kubernetes (kind), Semgrep (SAST), SonarQube, Wazuh (HIDS / SIEM), Elastic Stack (Elasticsearch, Logstash/Filebeat, Kibana), TheHive (SOAR / Incident Management).
9. Sprint 1: Business Logic & AI (Session 6)	Train and validate the local log anomaly detection model (Isolation Forest / Autoencoder) on baseline benign and attack traffic datasets. Package the inference logic into a standalone Python service with an API endpoint /score that consumes log events and returns anomaly classifications with confidence metrics.
10. Sprint 2: Integration & Frontend (Session 7)	Connect the AI inference pipeline to Elasticsearch and Wazuh alerts. Create dedicated Kibana SOC dashboards displaying live threat distributions, anomaly scores over time, top suspicious source IPs, and auto-populate actionable alerts into TheHive incident response workspace.
11. Sprint 3: Robustness & Security (Session 9)	Calibrate AI anomaly detection thresholds to minimize false positives (noise reduction). Enforce security hardening: container network isolation (devsecops-net), non-root container users, Kubernetes Secrets for sensitive credentials, API rate limiting, and input sanitization to prevent pipeline evasion.
12. Sprint 4: Cloud & Industrialization	Build and optimize multi-stage Dockerfiles for the app and AI microservice. Implement an automated multi-stage GitHub Actions CI/CD pipeline (Semgrep SAST → SonarQube quality gate → Docker build/push →

(Sessions 10-11)	Kubernetes deployment) with automated alert-to-case synchronization.
-------------------------	--